Chapter 2: Regression and Cross-Validation

Part II — Introduction to Statistical Learning

Omid Safarzadeh

Abstract

Linear regression is the first real learning algorithm, and the template for every one after it: choose a model, write down a loss, minimise it, then ask how well the fit will generalise. This chapter derives least squares and its closed-form solution, proves the two results that justify it — the normal equations and the Gauss–Markov theorem — decomposes prediction error into bias and variance, and shows how cross-validation estimates that error from the data alone. That last tool is how you choose model complexity before you ever touch a test set.

Contents

1 The linear model and least squares 1

2 Why least squares, and what error it makes 2

3 Estimating test error: cross-validation 3

1 The linear model and least squares

We model an output y as a linear function of inputs plus noise,

$$y = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p + \varepsilon, \qquad \text{or in matrix form} \qquad y = X\beta + \varepsilon,$$

with X the $n \times (p{+}1)$ design matrix (a column of ones for the intercept). **Least squares** picks the coefficients that make the residuals smallest in total square:

Definition 1.1 (Least-squares estimator)**.** $\hat{\beta} = \arg\min_\beta \ \mathrm{RSS}(\beta)$, where $\mathrm{RSS}(\beta) = \|y - X\beta\|^2 = \sum_{i=1}^{n} (y_i - x_i^\top \beta)^2$.

Setting the gradient to zero gives the **normal equations** $X^\top X\,\hat{\beta} = X^\top y$, so whenever $X^\top X$ is invertible,

$$\hat{\beta} = (X^\top X)^{-1} X^\top y.$$

For a single predictor this collapses to formulas you can read off directly: the slope is the covariance of x and y over the variance of x, and the line passes through the means,

$$\hat{\beta}_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}, \qquad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1\, \bar{x}.$$

Quality of fit is summarised by $R^2 = 1 - \mathrm{RSS}/\mathrm{TSS}$, the fraction of the variance in y the model explains, where $\mathrm{TSS} = \sum_i (y_i - \bar{y})^2$. In code it is all a few sums:

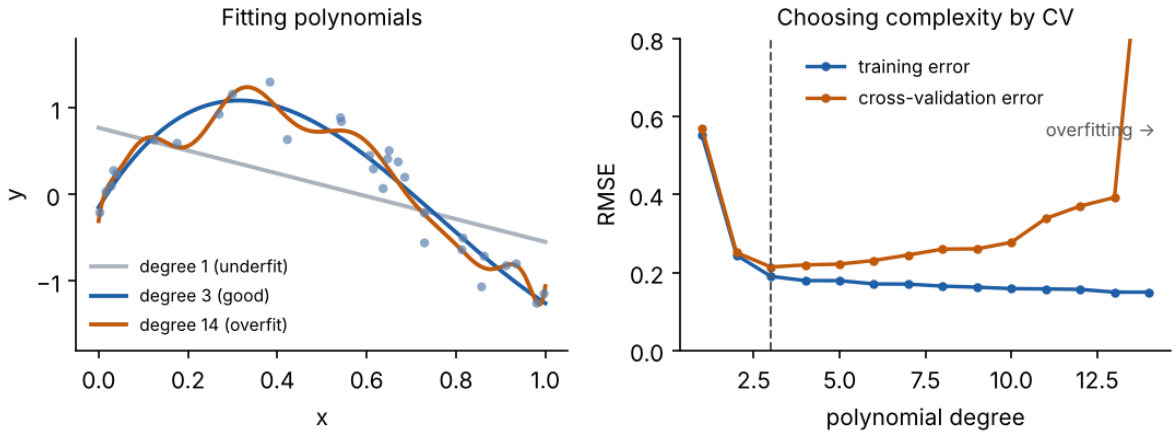
```python
def simple_linear_regression(x, y):
    n = len(x); xbar = sum(x)/n; ybar = sum(y)/n
    sxy = sum((xi-xbar)*(yi-ybar) for xi, yi in zip(x, y))
    sxx = sum((xi-xbar)**2 for xi in x)
    b1 = sxy / sxx                          # slope     = Sxy / Sxx
    b0 = ybar - b1 * xbar                   # intercept = ybar - b1*xbar
    return b0, b1

def r_squared(x, y, b0, b1):
    ybar = sum(y) / len(y)
    ss_res = sum((yi - (b0 + b1*xi))**2 for xi, yi in zip(x, y))
    ss_tot = sum((yi - ybar)**2 for yi in y)
    return 1 - ss_res / ss_tot

x = [1, 2, 3, 4, 5]; y = [2.1, 3.9, 6.1, 7.9, 10.2]
b0, b1 = simple_linear_regression(x, y)
print(round(b0, 3), round(b1, 3))         # -0.02 2.02
print(round(r_squared(x, y, b0, b1), 4)) # 0.9983
```

Multiple regression is the same idea with the matrix normal equations in place of the two sums. But a good fit to the data in hand is not the goal — a good fit to *new* data is. A flexible enough model can drive the training error to zero and still predict terribly, which is the whole problem the rest of the chapter is about.

The bias-variance tradeoff: training error falls, validation error turns back up



As the polynomial degree rises, the training error keeps falling while the cross-validation error turns back up: the model starts fitting noise. The sweet spot is the degree that minimises the validation curve.

2 Why least squares, and what error it makes

Two results justify least squares, and one identity explains its errors.

Theorem 2.1 (Normal equations)**.** $\hat{\beta}$ minimises RSS if and only if $X^\top X\,\hat{\beta} = X^\top y$.

Proof. $\mathrm{RSS}(\beta) = (y - X\beta)^\top (y - X\beta)$. Its gradient is $\nabla_\beta \mathrm{RSS} = -2X^\top (y - X\beta)$. Setting it to zero gives $X^\top X\beta = X^\top y$. The function is convex (its Hessian $2X^\top X$ is positive semidefinite), so the stationary point is a global minimum. □

Theorem 2.2 (Gauss–Markov)**.** Assume $E[\varepsilon] = 0$ and $\mathrm{Cov}(\varepsilon) = \sigma^2 I$. Among all *linear unbiased* estimators of β, the least-squares estimator has the smallest variance — it is BLUE (best linear unbiased estimator).

Sketch. Any linear estimator has the form $\tilde{\beta} = Cy$, and unbiasedness forces $CX = I$. Write $C = (X^\top X)^{-1}X^\top + D$. The constraint $CX = I$ then forces $DX = 0$. Expanding $\mathrm{Cov}(\tilde{\beta}) = \sigma^2 CC^\top$ and using $DX = 0$ gives

$$\mathrm{Cov}(\tilde{\beta}) = \sigma^2 (X^\top X)^{-1} + \sigma^2 DD^\top.$$

The second term is positive semidefinite, so no unbiased linear estimator beats least squares. □

Even the best unbiased fit still makes errors, and they split into three named pieces. For a test point x_0 with truth $f(x_0)$ and noisy outcome $y_0 = f(x_0) + \varepsilon$, the expected squared error of a fitted $\hat{f}$ is

Theorem 2.3 (Bias–variance decomposition)**.**

$$E\big[(y_0 - \hat{f}(x_0))^2\big] = \underbrace{\big(f(x_0) - E[\hat{f}(x_0)]\big)^2}_{\text{bias}^2} + \underbrace{\mathrm{Var}\big(\hat{f}(x_0)\big)}_{\text{variance}} + \underbrace{\sigma^2}_{\text{irreducible}}.$$

Proof. Condition on x_0 and add and subtract $E[\hat{f}(x_0)]$ inside the square; the cross terms vanish because ε is independent of $\hat{f}$ and $E[\hat{f}] - \hat{f}$ has mean zero. What remains is the squared bias, the variance of $\hat{f}$, and $\mathrm{Var}(\varepsilon) = \sigma^2$. □

A simple model has high bias and low variance; a flexible one has low bias and high variance. The validation curve in the figure is this trade-off made visible — and the irreducible σ^2 is the floor no model can cross.

3 Estimating test error: cross-validation

Training error is an optimistic estimate of test error — the model was chosen to fit that very data — so it cannot be used to choose complexity. **Cross-validation** estimates test error honestly by holding data out of the fit. In ***k*-fold** CV the data are split into k blocks; each block is scored once by a model trained on the other $k - 1$, and the k scores are averaged. **Leave-one-out** CV (LOOCV) is the $k = n$ extreme: each point is predicted by a model fit on all the others.

LOOCV looks like it needs n refits, but for a linear model it is free. Let h_{ii} be the ith diagonal of the **hat matrix** $H = X(X^\top X)^{-1}X^\top$ (the *leverage* of point i). Then

Theorem 3.1 (LOOCV shortcut)**.**

$$\mathrm{CV}_{\mathrm{LOO}} = \frac{1}{n}\sum_{i=1}^{n}\left(\frac{y_i - \hat{y}_i}{1 - h_{ii}}\right)^2,$$

where $\hat{y}_i$ is the fitted value from the *full* fit. No refitting is needed: one fit gives every leave-one-out error.

The choice of k is itself a bias–variance trade-off, now about the error *estimate*: LOOCV is nearly unbiased but its n fits are highly correlated, so the estimate has high variance; small k has the opposite. In practice $k = 5$ or $k = 10$ is the usual compromise. With the CV curve in hand, model selection is just "pick the complexity that minimises it" — the dashed line in the figure.

Summary

Least squares minimises $\|y - X\beta\|^2$, solved by the normal equations $\hat{\beta} = (X^\top X)^{-1} X^\top y$; Gauss–Markov makes it the best linear unbiased estimator. Its expected error splits into $\text{bias}^2 + \text{variance} + \sigma^2$, and flexibility trades the first two off. Training error cannot measure this trade-off, but cross-validation can — k-fold or the no-refit LOOCV shortcut $\frac{1}{n}\sum((y_i - \hat{y}_i)/(1 - h_{ii}))^2$ — which is how you choose model complexity from data alone.